CS300 Project One Analysis

Jin Han Jun

Big O Runtime Analysis

Code	Line Cost	# Time Executes	Total Cost
Create Vector	1	1	1
Open fileName	1	1	1
If fileName cannot open	1	1	1
While there are lines in file	1	N+1	N+1
Read next line	1	n	n
Split line at each comma and store values in tokens	1	n	n
If tokens contain less than 2 values	1	n	n
Create Course course	1	n	n
Course.courseNumber = tokens[0]	1	n	n
Course.name = tokens[1]	1	n	n
For each token starting at tokens[2]	1	N(k+1)	N(k+1)
Add token to course.prerequisites	1	N x k	N x k
Add course to courses	1	n	n
Close file	1	1	1
For course in courses	1	N+1	N+1
For prerequisite in course.prerequisites	1	N(k+1)	N(k+1)
Set prerequisiteFound to false	1	N x k	N x k
For storedCourse in courses	1	Nk(n+1)	Nk(n+1)
If storedCourse.courseNumber equals prerequisite	1	N x k x n	n^2k
Set prerequisiteFound to true	1	N x k	nk
break	1	N x k	nk
If prerequisiteFound equals false	1	N x k	nk
Return courses	1	1	1
Total Cost			2n^2k + 8nk + 11n + 7
Runtime			O(n^2k)

Hash Table Runtime Analysis

Code	Line Cost	# Times Executes	Total Cost
Create HashTable courses	1	1	1
Open fileName	1	1	1
If fileName does not open	1	1	1
While lines in file	1	N + 1	N + 1
Read next line	1	n	n
Split line at comma and store value in tokens	1	n	n
If tokens has less than 2 values	1	n	n
Create Course course	1	n	n
Course.courseNumber = tokens[0]	1	n	n
Course.name = tokens[1]	1	n	n
For each token starting at tokens[2]	1	N(k+1)	N(k+1)
Add token to course.prerequisites	1	N x k	nk
Insert course in courses using course.courseNumber as key	1	n	n
Close file	1	1	1
For each course in courses	1	N+1	N+1
For each prerequisite in course.prerequisites	1	N(k+1)	N(k+1)
If courses.Search(prerequisite) not found	1	N x k	nk
Return courses	1	1	1
Total cost			11n + 4nk + 7
Runtime			O(nk) , O(n) if k constant

Worst Case Runtime

O(n^2k)

Assuming the hash table has too many collisions, insertion and searching courses takes O(n) instead of O(1) time. Because course prerequisites are checked, O(n^2k) is the worst case runtime.

Binary Search Tree Runtime Analysis

Code	Line Cost	# times executes	Total cost
Create BinarySearchTree courses	1	1	1
Open fileName	1	1	1
If file cannot be opened	1	1	1
While there are lines in file	1	N+1	N+1
Read next line	1	n	n
Split line at each comma and store values in tokens	1	n	n
If tokens contain less than 2 values	1	n	n
Create course object	1	n	n
Set course.courseNumber to tokens[0]	1	n	n
Set course.name to tokens[1]	1	n	n
For each token beginning at tokens[2]	1	N(k+1)	N(k+1)
Add token to course.prerequisites	1	N x k	nk
Insert course into courses using course.courseNumber	O(n)	n	O(n^2)
Close file	1	1	1
For each course in courses	1	N+1	N+1
For each prerequisite in course.prerequisites	1	N(k+1)	N(k+1)
If courses does not contain prerequisite	O(n)	nk	O(n^2k)
Return courses	1	1	1
Total cost			O(n^2 + n^2k +n +nk)
runtime			O(n^2k)

Worst Case Runtime

O(n^2k)

In the case where the binary search tree is unbalanced, insertion and searching courses takes O(n) time instead of O(log n). Because the program checks each courses' prerequisites, O(n^2k) is the worst case runtime.

Data Structures

Vectors allow for courses to be stored in a list. The good thing about vectors is that the courses are accessed by their position in the vector in constant time. Something to keep in mind is that if we were to search a specific course then it would take O(n) time as the program checks every course in the vector.

Hash tables use keys based on the courses' course number. The good thing about hash tables is that if we were to search for a specific course then it would take O(1) time which is faster than a vector in this scenario. Something to keep in mind is that hash tables do not store courses alphanumerically so in order to sort the courses out, the courses would need to be implemented into a different structure to be sorted.

A binary search tree stores courses based on courses' course number. The good thing about a binary search tree is searching for a course takes O(log n) time. However, if the tree is unbalanced then it will change the searching to O(n) time.

Recommendation

I recommend a binary search tree for the implementation of the course planner. It allows for courses to be printed alphanumerically through in order traversal without the need to separately sort. I think the only downside to a binary search tree other than the unbalancing case is that binary search trees are more difficult to implement and instantiate compared to vectors or hash tables. The course planner should display courses based on order and be able to search for specific courses. A binary search tree is sufficient for these purposes and circumstances.